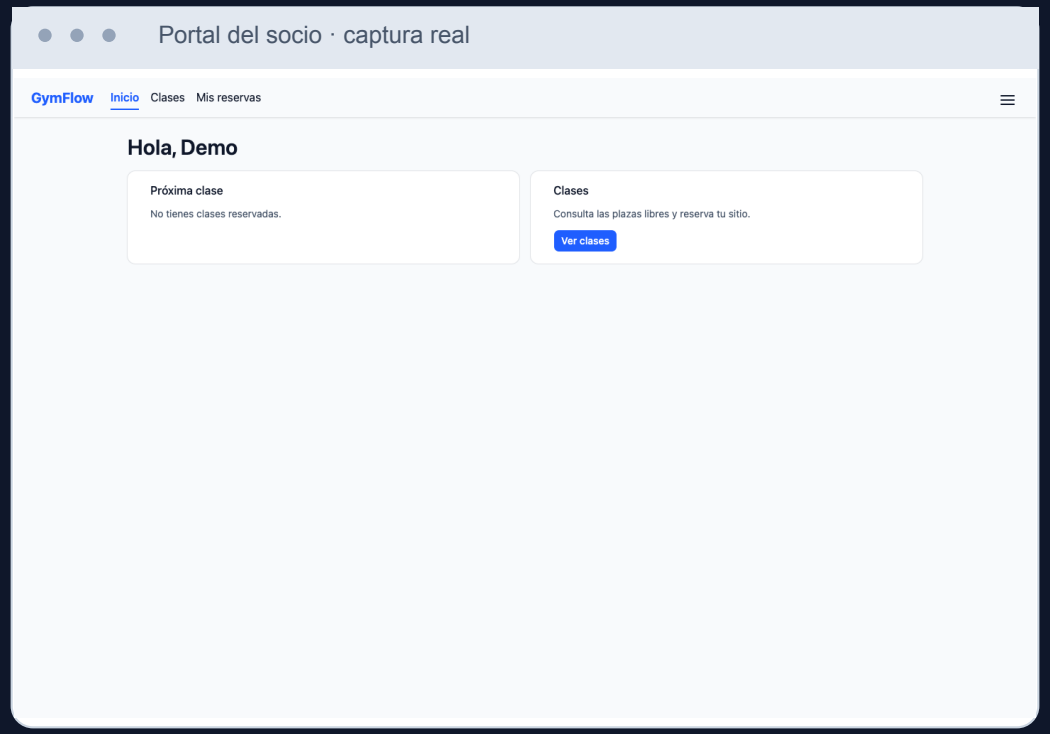
GYMFLOW

PROYECTO G1 · 20 OCT 2026

Menos gestión. Más movimiento.

La operación del gimnasio,
en un mismo lugar.



Negocio y equipo 00–08 min

Solución técnica 08–20 min

Una operación conectada, sin registros dispersos

01

SITUACIÓN

Información fragmentada

Clientes, clases y pagos en registros separados.

02

NECESIDAD

Reservas confiables

Saber quién puede reservar y cuántas plazas quedan.

03

RESPUESTA

Una fuente compartida

Una API y un portal para coordinar la operación.

Valor buscado: menos errores operativos y más autonomía para el socio.

Tres perfiles, una experiencia coordinada

CLIENTE

Reserva con autonomía

Consulta clases, sus reservas, sus pagos y su perfil.

ENTRENADOR

Gestiona sus sesiones

Organiza sus clases y las reservas asociadas.

ADMINISTRACIÓN

Controla la operación

Gestiona clientes, clases, pagos, cuentas y CSV.

Cada persona accede a la información que necesita para su función.

Del acceso a una plaza confirmada

Acceder
Cuenta del socio

→

Elegir
Clase y disponibilidad

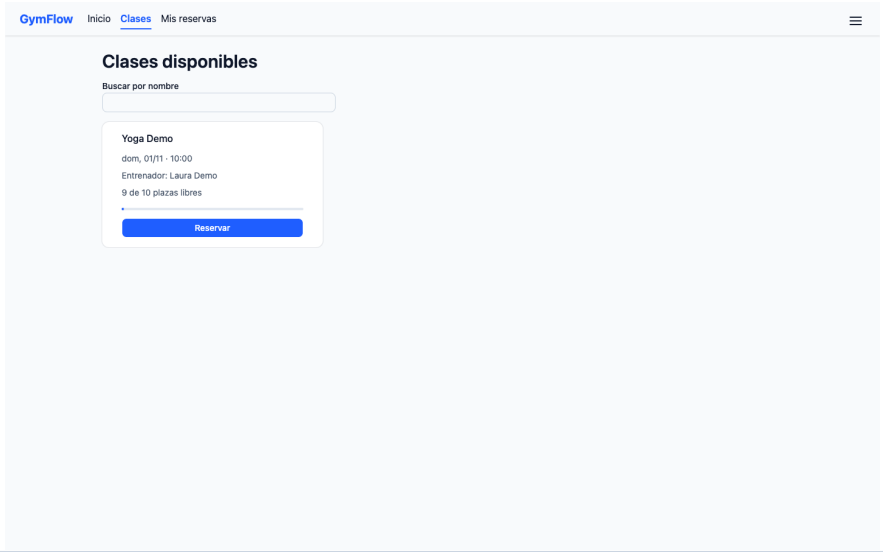
→

Reservar
Validación del gimnasio

→

Consultar
Mis reservas

Catálogo real · entorno local



DEMO · 60 SEGUNDOS

Entrar → elegir clase → reservar

Cliente activo + pago
+ plaza disponible.

Abrir demo local

Cinco responsabilidades, una entrega compartida

1	2	3	4	5
Miguel	**Juan Carlos**	**Fabiana**	**Brian**	**David**
Product Owner	Scrum Master	Backend	Frontend	DevOps
00:00–04:00	**04:00–08:00**	**08:00–12:00**	**12:00–16:00**	**16:00–20:00**

La participación y las preguntas se distribuyen entre todo el equipo.

De una necesidad a una historia verificable

HISTORIA DE USUARIO

«Como cliente, quiero reservar una clase para asegurar mi plaza».

Negocio → regla observable → tarea

1 **Acceso**

Cuenta activa y autenticada.

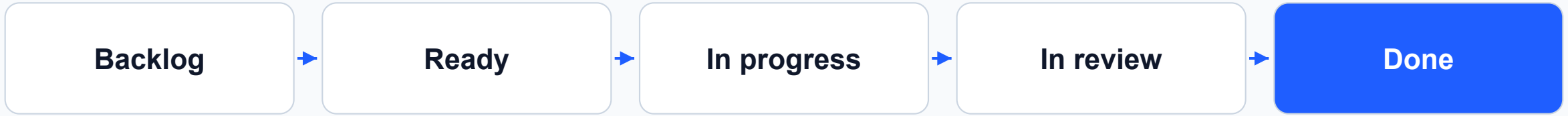
2 **Admisión**

Cliente activo, pago y plaza libre.

3 **Resultado**

Reserva propia, sin duplicados.

Priorizar y desbloquear antes de desarrollar



DEPENDENCIAS REALES

#105 · Autenticación
Proteger recursos con JWT

→

#106 · Autorización
Limitar por rol y propietario

Responsable claro
Prioridad visible
Bloqueos explícitos

Abrir tablero del equipo

Milestones por nivel de entrega; historias divididas en tareas.

Una tarea termina cuando se puede comprobar

01 **Especificación**
Requisitos y escenarios antes del código.

02 **Implementación**
Capas y reglas acordadas del proyecto.

03 **Verificación**
Lint, tests y validación de OpenSpec.

04 **Revisión**
Pull request y aprobación humana.

IA para acelerar. Equipo para decidir.

OpenSpec
Bitácora de decisiones
Revisión del resultado

El negocio no depende del framework

interfaces/

Rutas, controladores y schemas

HTTP / Pydantic

↓

application/

Casos de uso y reglas

GymUseCases

↓

domain/

Entidades y puertos

dataclasses / Protocol

infrastructure/

SQLAlchemy
Repositorios
MySQL
Adaptadores de seguridad

Implementa los puertos del dominio.

Python 3.14 · FastAPI · Pydantic · SQLAlchemy · Alembic

Una identidad; dos vínculos de negocio

MySQL · entidades separadas · relaciones explícitas · integridad por claves



Reservas resuelve Cliente ↔ Clase (N:N).

UNIQUE evita duplicados · FK RESTRICT protege el historial.
versions registra la versión del esquema.

Autenticar no basta: hay que delimitar el acceso

Login
Email + contraseña

→

JWT
Firma + caducidad

→

Cuenta actual
Activo, rol y vínculo

→

Autorización
Operación + propietario

Admin	Gestión completa y exportación CSV
Staff	Solo sus clases y sus reservas asociadas
Cliente	Solo sus datos y sus propias reservas

Argon2id para contraseñas · JWT configurable · 401 / 403 / 404 según el caso

La última plaza se decide en una transacción

EXTRACTO REAL · GymUseCases._validate

```
clase = self._require(Clase, entity.id_clase, lock=True)

if cliente.estado_membresia != "Activo":
    raise ConflictError(
        "Solo los clientes activos pueden reservar"
    )

if ocupacion >= clase.capacidad_max:
    raise ConflictError("La clase no tiene plazas disponibles")
```

Extracto con líneas intermedias omitidas: también valida pago y unicidad.

BLOQUEO DE LA CLASE

Dos peticiones. Una plaza libre.

201 · reserva creada

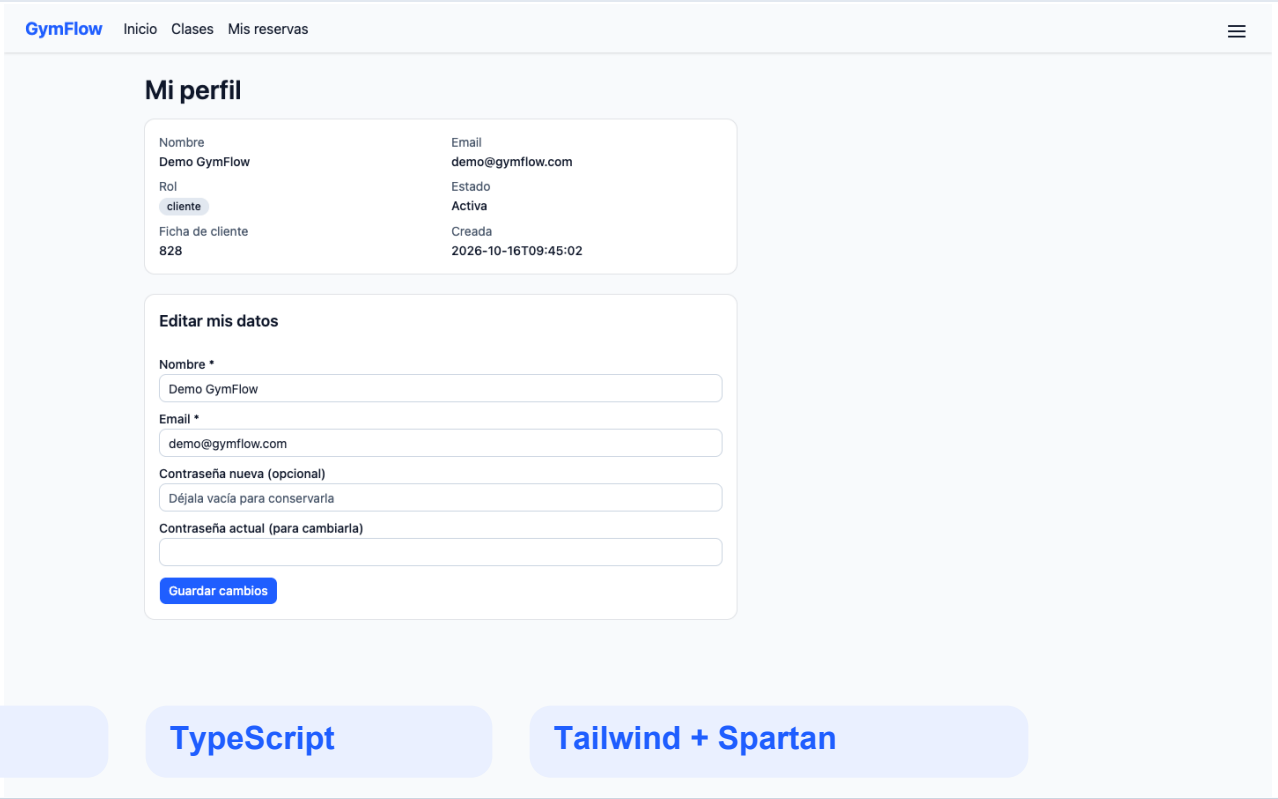
409 · sin plaza

SQLAlchemy usa with_for_update().

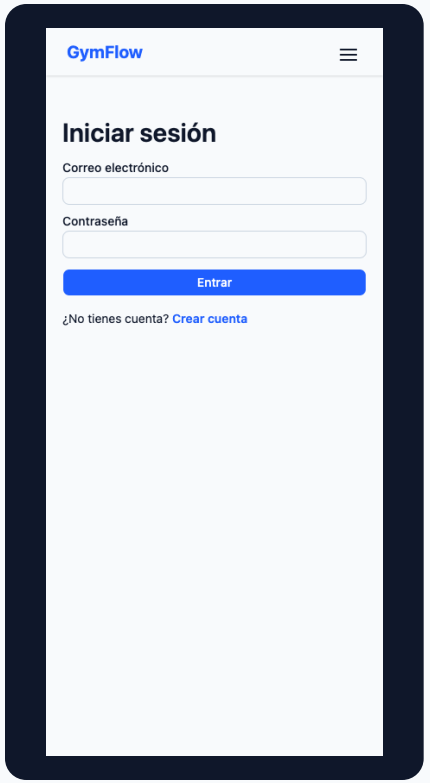
Naming: snake_case en funciones/variables · PascalCase en clases · CONSTANTES

Una interfaz que hace visible el estado del gimnasio

Perfil real · Angular conectado a la API



Angular 22 **TypeScript** **Tailwind + Spartan**



Componentes reutilizables.
Diseño adaptable.

De la pantalla a la API, con un contrato compartido

Página / componente
Interacción y formulario

↓

Servicio de recurso
Contrato y llamada HTTP

↓

Interceptor
Bearer JWT y respuesta 401

↓

API /v1
Reglas y permisos reales

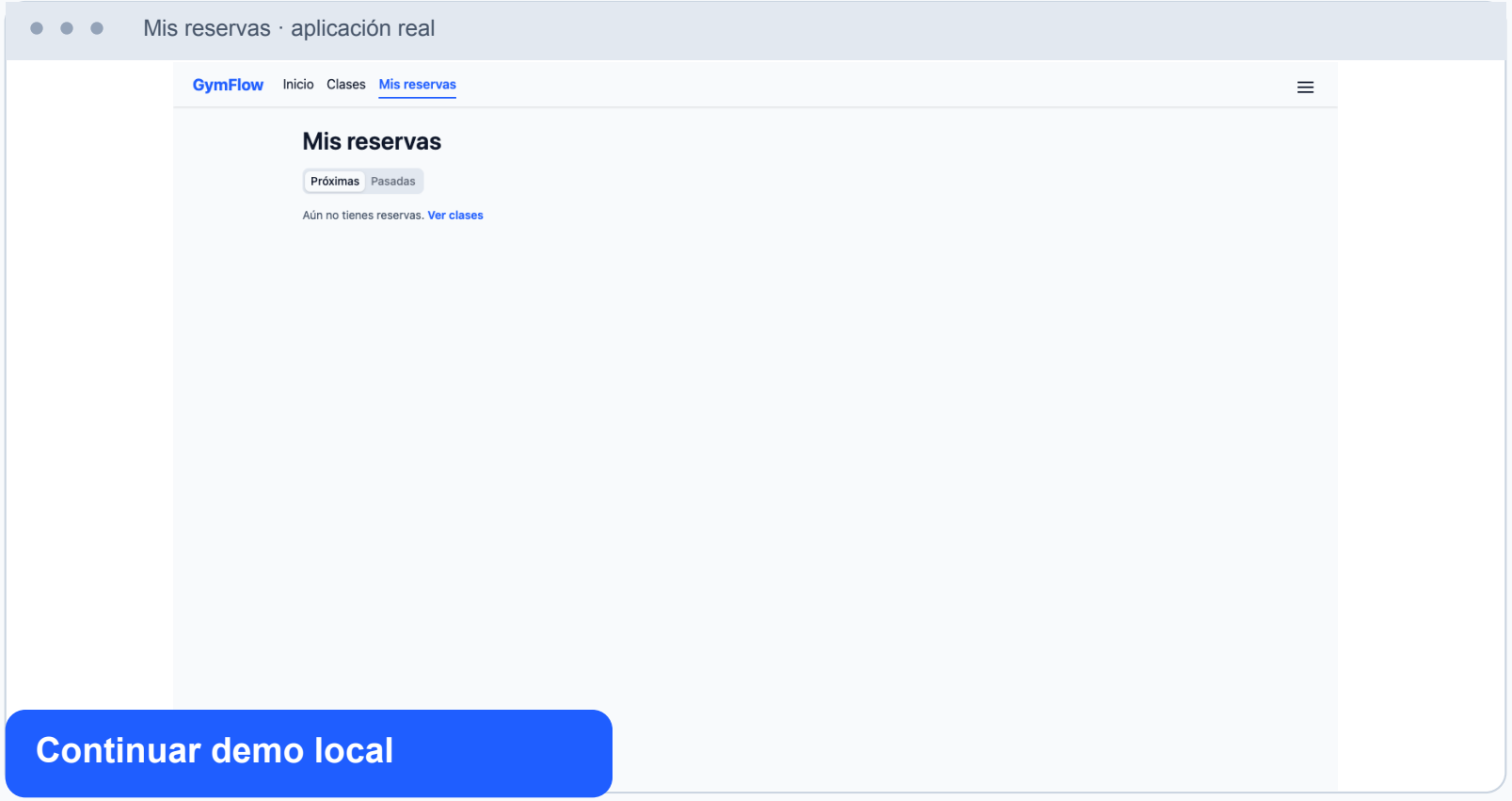
auth.interceptor.ts · extracto real

```
const token = leerToken();
const esApi = req.url.startsWith(RUTA_API);
const peticion =
  token && esApi
    ? req.clone({
        setHeaders: { Authorization: `Bearer ${token}` }
      })
    : req;
```

La interfaz orienta. La API autoriza.

Un 401 con token elimina la sesión local.
Cupo agregado: disponibilidad sin reservas ajenas.

La demo termina donde el usuario espera



01 Consultar clases

02 Crear la reserva

03 Ver Mis reservas

04 Cancelar y confirmar

05 Comprobar el cupo

Los errores también forman parte de la experiencia

401

Sesión

Volver a identificarse.

409

Conflicto

Sin plaza o reserva duplicada.

422

Validación

Corregir los datos del formulario.

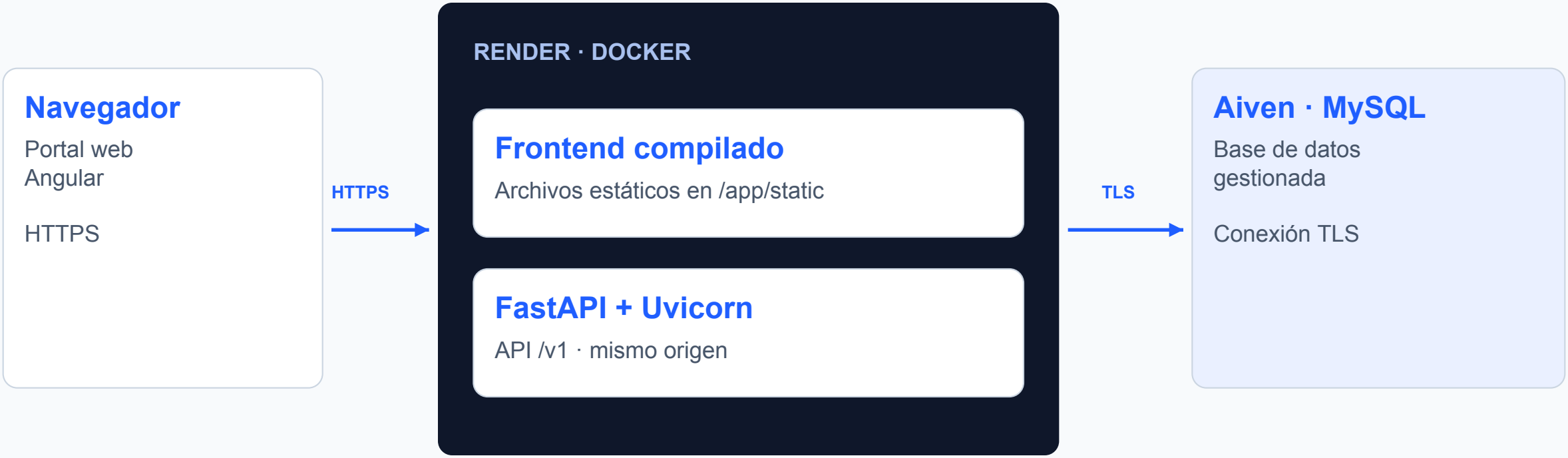
Red

Conexión

Mostrar un mensaje y reintentar.

117 pruebas de frontend · 26 suites · verificación local del 19/10

Una aplicación desplegable, con datos persistentes



En local: Docker Compose coordina MySQL, API y Nginx.

Configuración versionada en Dockerfile, docker-compose.yml y render.yaml.

Del pull request a una entrega repetible

Issue
Necesidad y AC

→

feature/…
Cambio aislado

→

Pull request
CI + revisión

→

main
Integración

↓

Render
Auto-deploy por commit

GitHub Actions

Ruff + pytest con MySQL real.
PR a main con revisión humana.

render.yaml · real

```
runtime: docker
plan: free
branch: main
autoDeployTrigger:
  commit
```

Commit convencional

feat(auth): configura la duración del JWT

Calidad y documentación que se pueden abrir

376

Pruebas backend

CI de main · 19/10

117

Pruebas frontend

26 suites · local · 19/10

API

Swagger / ReDoc

Ejecución

README + Taskfile

Datos

ER + diccionario

Decisiones

OpenSpec + bitácora

Healthchecks: /health y /health/db · logs de aplicación · esquema versionado

Un gimnasio conectado.
Un equipo responsable.

Orden

Información centralizada

Confianza

Permisos y reglas de negocio

Continuidad

Ejecución y entrega repetibles

Ver repositorio **Abrir aplicación local**

Preguntas por área: Miguel · Juan Carlos · Fabiana · Brian · David

Reloj de ensayo y relevos

20 minutos exactos · cinco turnos de 4 minutos · anexos fuera del pase

Miguel	Product Owner	**00:00–04:00**	Diapositivas 1–4
Juan Carlos	Scrum Master	**04:00–08:00**	Diapositivas 5–8
Fabiana	Backend	**08:00–12:00**	Diapositivas 9–12
Brian	Frontend	**12:00–16:00**	Diapositivas 13–16
David	DevOps	**16:00–20:00**	Diapositivas 17–20

Señal de relevo a 3:40 · cambio de ponente a 4:00 · demo sin búsquedas improvisadas

Preparar una demo que no dependa de improvisar

Entorno

task up · o task docker:up

Datos

Cuenta demo, cliente activo y pago registrado.

Clase

Futura, con plaza libre y sin reserva duplicada.

Pestañas

Portal · Mis reservas · Swagger · código · CI

VERIFICACIÓN DEL 19/10

Local: esquema 0.4
Público: esquema 0.3

Comprobar paridad antes de usar Render para mostrar permisos actuales. La demo preparada usa local.

Plan B: capturas incluidas + explicación del flujo. No prolongar la espera más de 15 s.

Distinguir la entrega actual de las siguientes mejoras

IMPLEMENTADO EN MAIN

Operación y acceso seguros

CRUD · filtros · paginación · CSV
JWT · roles · perfil y contraseña
Portal Angular · Docker · CI

EVOLUCIÓN DEL BACKLOG

Integraciones y seguridad avanzada

Cobro online y notificaciones
OAuth social / 2FA / refresh tokens
Caché y actualizaciones en tiempo real

Regla vigente: al menos un pago; el modelo no define periodos de membresía.

Fuentes verificables y preguntas repartidas

Base: main · commit 2beb0b4 · revisión del 19 de octubre de 2026

Repositorio y código

Abrir fuente

Tablero de coordinación

Abrir fuente

CI backend · 376 pruebas

Abrir fuente

Documentación y decisiones

Abrir fuente

Esquema relacional

Abrir fuente

Miguel

Problema, objetivos y producto

Juan Carlos

Historias, prioridades y proceso

Fabiana

API, permisos y modelo de datos

Brian

Interfaz y experiencia de usuario

David

Docker, nube, CI y operación